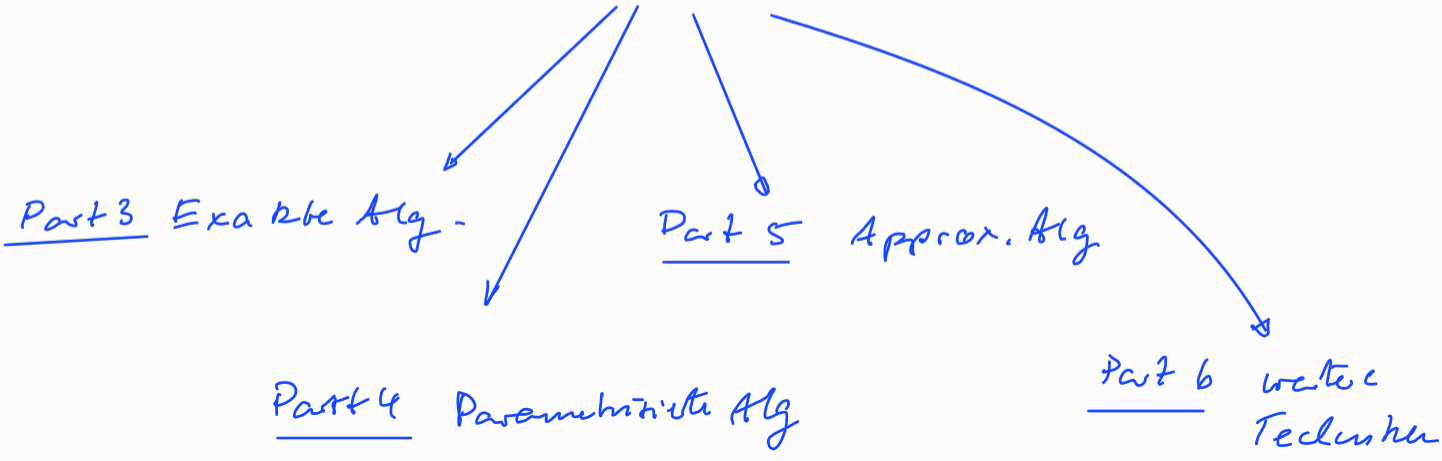
Fahrplan

Part 1 TM, Algorithmen & Entscheidbarkeit

Part 2 Einfach vs schwere Probleme & NP vollständigkeit



Part 3 Exakte Alg.

Part 4 Parameterisierte Alg

Part 5 Approx. Alg

Part 6 weitere Techniken

2.1 "Einfache" vs "schwere" Probleme

Grundidee: wollen klassifizieren ob Problem "einfach" oder "schwer" ist

einfach heißt i.d.R. es existiert ein Alg. der das Problem in Polyn. zeit $O(n^k)$ löst.

n = input größe
k = konstante

wdh (asymptotische Schranken)

$$O(g(n)) = \{ f(n) : \exists \text{ konstanten } c > 0, n_0 > 0 \text{ sd } \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n) \}.$$

$$\Omega(g(n)) = \{ f(n) : \exists \text{ konstanten } c > 0, n_0 > 0 \text{ sd } \forall n \geq n_0 : 0 \leq c \cdot g(n) \leq f(n) \}$$

$$\Theta(g(n)) = \{ f(n) : \exists \text{ konstanten } c_1, c_2 > 0, n_0 > 0 \text{ sd } \forall n \geq n_0 : 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 g(n) \}$$

$(f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n)) \;\&\; f(n) \in \Omega(g(n))$.

Betrachten jetzt ein paar probleme, die algorithmisch einfach sind, d.h. in polyn. zeit gelöst werden können

- kürzester weg in graphen
- 2SAT
- Eulertouren

Anmerkung: zu allen problemen gibt "minimale" abwandlungen für die bisher kein polyzeit Alg. gefunden wurde.

1. Kürzeste Wege in Graphen

wir betrachten ungerichtete & gerichtete Graphen.

Defs: $G = (V, E)$ mit endlicher Menge V

"ungerichtet": $E \subseteq \binom{V}{2}$ = Menge aller 2-elementige Mengen von V

"gerichtet" $E \subseteq V \times V$

ungerichtet kann als Spezialfall von gerichtet betrachtet werden, einfach alle $\{u,v\} \in E$ ersetzen durch (v,u) & (u,v).

Um Schreibweise zu vereinfachen & wir manchmal nicht unterscheiden zw. gerichtet & ungerichtet schreiben wir uv für Kanten $\{uv\}$ bzw. (uv).

(G, w) ist ein Graph mit gewichteter $w: E \to \mathbb{R}$
im einfachsten Fall $w(e) = 1 \quad \forall e \in E$.

Ein Pfad (oft auch Weg) in $G = (V, E)$ ist eine Sequence $P = (v_1 \dots v_k)$, $k \geq 1$ von Knoten od $v_i v_{i+1} \in E$, $1 \leq i \leq i-1$

P wird dann auch $v_1 v_k$ – Pfad genannt.
& v_k erreichbar von v_1 genannt.

$P = (v_1 .. v_k)$ ist „einfach" wenn $v_i \neq v_j \; \forall i \neq j$.

Länge von P ist $\sum_{i=1}^{k-1} w(v_i v_{i+1}) =: \ell(P)$

distanz zweier knoten u, v in G ist

$$d_G(u,v) = \min_{\substack{P \text{ ist} \\ uv\text{-pfad}}} \ell(P).$$

wenn es kein uv-Pfad gibt, dann $d(uv) = \infty$.

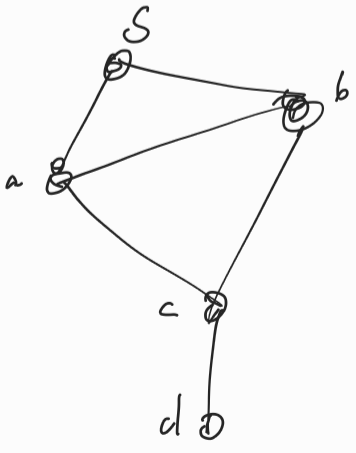
Ziel: Bestimme f.a $u, v \in V$ deren distanz $d(uv)$.

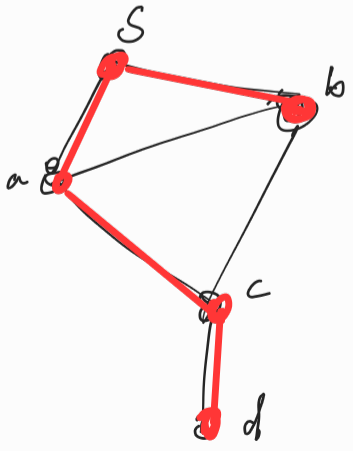
Q: Wie kann man diese in (G, w) bestimmen?
A: Abhg von Gewichtsfunktion w!

Annahme $w(e) = 1 \; \forall e \in E$.

dann gibt es verschiedene Alg. die das problem lösen können (Breitensuche, Dijkstra, Floyd-Warshall, Bellmann Ford)

BFS





Reihenfolge in der knoten via BFS besucht: s a b c d

init: $d(s,v) = \infty \quad \forall v$
$d(s,s) = 0$

besuche $a \to d(s,a) = \infty \Rightarrow d(s,a) = d(s,s) + 1 = 1$
(s schon besucht)

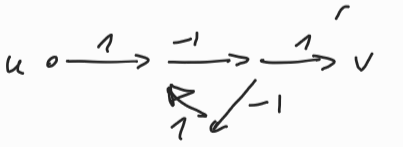
$b \to d(s,b) = \infty \to d(s,b) = d(s,s) + 1 = 1$

$c \to d(s,c) = \infty \longrightarrow d(s,c) = d(s,a) + 1 = 2$
(c als Nachbar von a (s schon besucht))

$d \to d(s,d) = \infty \longrightarrow d(s,d) = d(s,c) + 1 = 3$
(d als Nachbar von c)

das kann über alle v als Abbruchkriterium machen
& berechnet $d_0(u,v) \; \forall u,v \in V$ in polyn. Zeit.

was wenn wir jetzt negative Gewichte erlauben (Alg immer noch wenn keine "negativen" Kreise = "konservative" Gewichtsfkt (Bellman Ford)



$\Rightarrow \quad d(u,v) = \infty$ ... mmh ...

wir könnten uns auf einfache Pfade beschränken
dann im Bsp $d(u,v) = 1$

ABER: es gibt bislang kein polyzeit Alg. der dieses Problem löst!
$\Rightarrow$ gibt es einen?

Erfüllbarkeit Boolsche Formeln

geg: Boolsche Formel φ bestehend aus:

- Operatoren
  - UND $\wedge$
  - ODER $\vee$
  - NICHT $\neg$, $\bar{}$
- Literal = Boolsche Variable oder deren negation
- Klammern $(,)$

Bsp:

$$\varphi = \neg(x \vee y) \vee (z \wedge w \wedge \bar{u})$$

de morgan

$$= (\bar{x} \wedge \bar{y}) \vee (z \wedge w \wedge \bar{u})$$

Distribl.

$$= (\bar{x} \vee z) \wedge (\bar{x} \vee w) \wedge (\bar{x} \vee \bar{u}) \wedge (\bar{y} \vee z) \wedge (\bar{y} \vee w) \wedge (\bar{y} \vee \bar{u})$$

$$= C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5 \wedge C_6$$

$\hat{=}$ KNF, d.h. φ ist Konjunktion (UND) von Klauseln

Klauseln = Disjunktion (OR) von Literalen.

Übung: jedes φ kann in konjunktiver Normalform (KNF) geschrieben werden.

im obigen Bsp ist $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_6$ in KNF

od $|C_i| = 2$, d.h. besteht immer aus 2 Literalen,

das ist aber nicht immer so.

also $C_i = (x_1 \vee \bar{x}_2 \vee x_3 \vee x_4 \dots)$ möglich.

Frage: Gibt es Belegung der Variablen auf 0 (False) 1 (TRUE)

so dass φ TRUE wird?
(dann heißt φ erfüllbar (satisfiable)).

Letzteres ist bekannt SAT-Problem
als

k-SAT: $\varphi = C_1 \wedge \cdots \wedge C_m$ so dass $|C_i| = k \ \forall i$.

Bsp:

$\wedge$-SAT: $\varphi = x_1 \wedge \overline{x_2} \wedge x_4$ erfüllbar? JA x_1 1, x_2 0, x_3 1

$\varphi = x_1 \wedge \overline{x_2} \wedge x_2$ NEIN: weil $x_2 \neq \overline{x_2}$.

(einfach zu lösen, einfach zu checken ob Variable & deren Negation vorkommt, wenn nicht $\Rightarrow$ erfüllbar.

2-SAT? 3-SAT? ....

Wir zeigen daß 2-SAT in polynomieller Zeit lösbar ist.

Zuvor brauchen wir noch folgende Definitionen.

Betrachte gerichtete Graphen $G = (V, E)$ $E \subseteq V \times V$

od $(v,v) \notin E \; \forall v \in V$

(keine Schlingen)

Eine starke Zushgs. Komponente (SZK) Z von G

ist inklusions-max. $Z \subseteq V$ od es einen

uv-Pfad

& vu-Pfad $\forall u,v \in V$

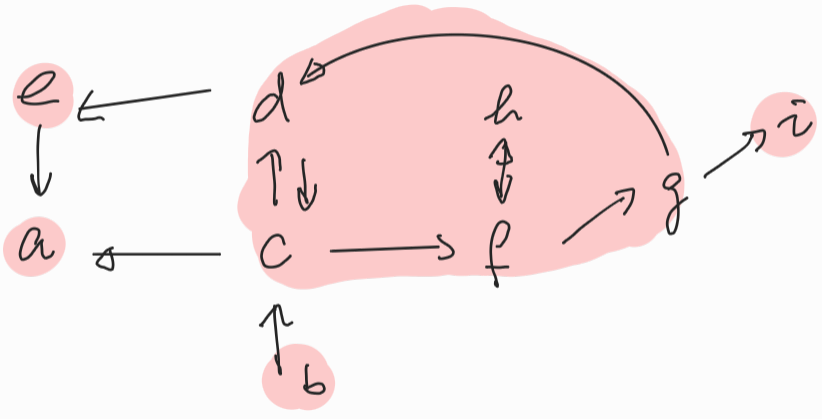
gibt.

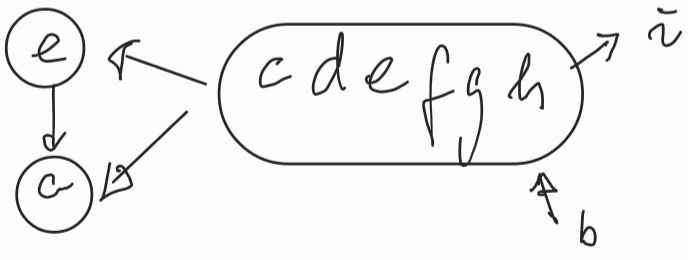
Analog. def: $u \overset{*}{\longleftrightarrow} v$ gdw $\exists$ uv-Pfad

& vu-Pfad

ist Äquivalenzrelation (refl, sym. trans.)

& Äquiv. klassen sind die SZK.





"Quotientengraph" Knoten: SZK

Kanten: $Z_i \to Z_j$ wenn $\exists u \in Z_i, v \in Z_j$

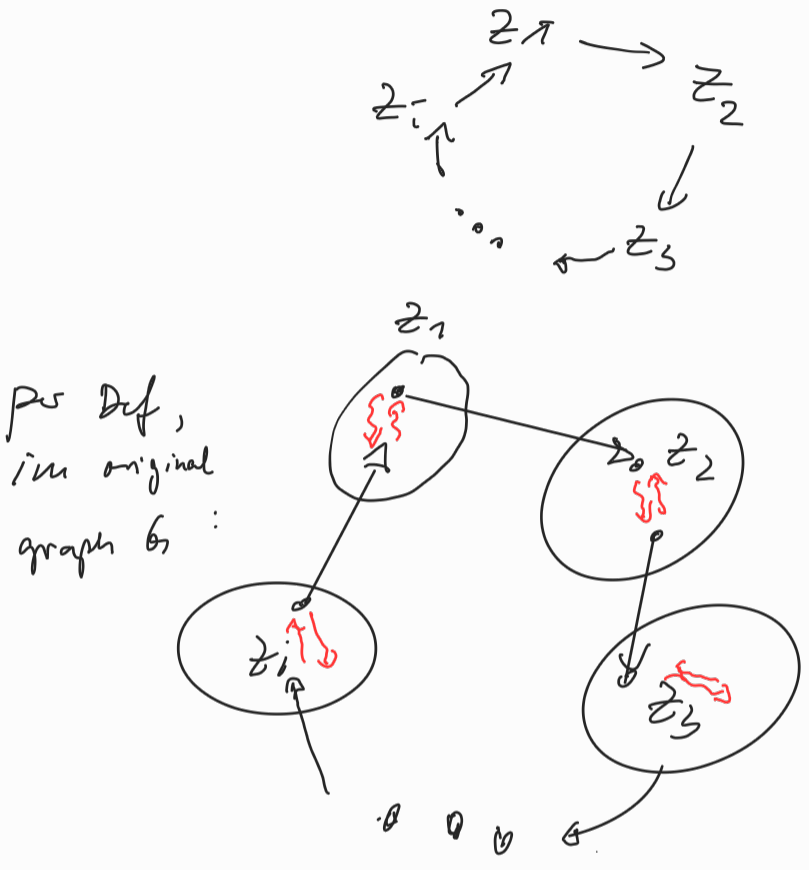
$i \neq j$ od $uv \in E$

$G / \overset{*}{\longleftrightarrow}$

Lemma: Quotientengraph $G/\overset{*}{\leftrightarrow}$ ist kreisfrei, d.h. existiert keine Teilgraphen der Form:

$v_1 \leftarrow\!\!\rightarrow v_2 \cdots \rightarrow v_k$

Beweis: Ann. existiert π $G/\overset{*}{\leftrightarrow}$



per Def, im original graph G:

$\lightning$ zu z_i inkl. max!

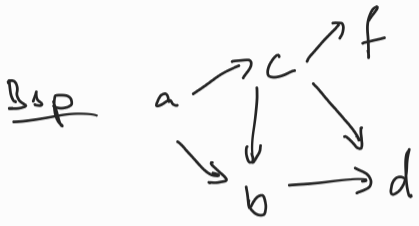
SZK in G können mittels Alg von Tarjan oder Alg. von Kosaraju in linearer Zeit bestimmt werden (Übung).

kreisfreie gerichtete graphen heißen DAG (directed acyclic graph).

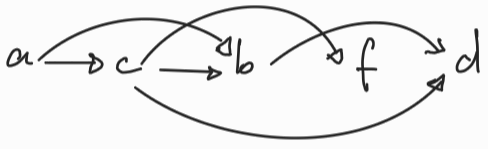
topologische Ordnung $<$ auf V, wenn

$v_i \longrightarrow v_j \in E \Rightarrow v_i < v_j$ erfüllt ist.

Bsp



$a < c < b < f < d$ wäre eine mögliche top. sortierung



Kanten zeigen von „rechts nach links".

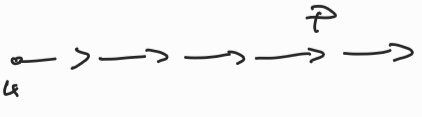
Lemma: G ist DAG $\Leftrightarrow$ es gibt topolog. sortierung von G

„$\Leftarrow$" G hat topol. sortierung.

Fall G kreis hat  gibt es offensichtlich keine topol. sortierung $\Rightarrow$ G DAG.

„$\Rightarrow$" • jeder DAG hat mind einen Quellknoten, d.h. einen knoten u od $vu \notin E$ $\forall v \in V$

(nimm längsten einfachen Pfad P in G.



wenn u keine Quelle $\Rightarrow \exists$ vu kante.

$\Rightarrow v \in P$ oder $v \notin P$.



$v \notin P$ ⇒ P wählt längsten Pfad ↯ $v \in P$ ⇒ G kein DAG ↯

Analog zeigt man, daß jeder DAG G auch eine Senke hat, d.h. einen Knoten u

mit $uv \notin E \ \forall v \in V$.

Simpler Alg für top. Sortierung für $G = (V, E)$

1. $L = \emptyset$ leere Liste (später geordnete Liste)
2. WHILE $V \neq \emptyset$
   1. Finde Quellknoten u in G
   2. lösche u aus V & betrachte „$G - u$"
   3. Füge u der Liste bei
3. RETURN L.

Laufzeit (in Aufg).

$L = (v_1 \dots v_n)$ top. sort. d.h.

Annahme es gibt kante uv. in G

⇒ v kann kein Quellknoten sein, sofern u nicht gelöscht wurde

⇒ u vor v gelöscht

⇒ u vor v in L

□

Thm 2-SAT ist in linearer Zeit lösbar

Beweis: φ in KNF, d.h. $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m$
sd $|C_i| = 2$ (besteht aus genau 2 Literalen).

OBdA: φ enthält keine Klauseln der Form $(x_i \vee \bar{x}_i)$ *
da diese immer wahr sind.

alle Klauseln C_i sind der Form $(l_i \vee l_j)$
mit Literalen l_i & l_j sd $l_i \neq \overline{l_j}$
*

Beobachtung: $(l_i \vee l_j) \overset{\text{equ.}}{\iff} (\neg l_i \Rightarrow l_j) \iff (\neg l_j \Rightarrow l_i)$

0	0	0	1	1	0	0
0	1	1	1	1	1	1
1	1	0	1	0	1	0
1	1	1	1	0	1	1

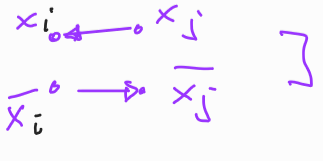
[Bsp $(x_i \vee \bar{x}_j) \iff \bar{x}_i \Rightarrow \bar{x}_j \iff x_j \Rightarrow x_i$]

$\Rightarrow$ φ kann als Konjunktion von Implikationen geschrieben werden.

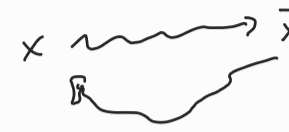
$\Rightarrow$ das können wir als gerichteten Graphen $G_\varphi = (V, E)$ darstellen mit

V = Menge der Variablen & ihren negationen

Für alle Klauseln $(l_i \vee l_j)$ fügen wir gerichtete Kanten $(\bar{l}_i, l_j)$ & $(\bar{l}_j, l_i)$ hinzu.

[Bsp $(x_i \vee \bar{x}_j) \rightarrow$ ]

Zeigen jetzt: φ ist erfüllbar $\Leftrightarrow$ keine Variablen x & $\bar{x}$ liegen in derselben starken Zusammenhangskomponente von $b\varphi$.

„$\Rightarrow$" Ann.  existiert.

Wenn x = true $\Rightarrow$ $\bar{x}$ = False

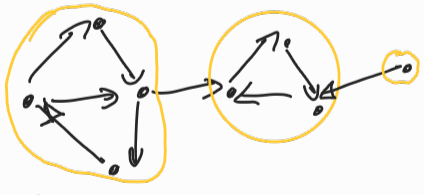
& hatte von Implikationen

$x \Rightarrow \dots \Rightarrow \bar{x}$ sagt dass φ nicht erfüllbar

$\Rightarrow$ x = FALSE $\Rightarrow$ $\bar{x}$ = TRUE $\Rightarrow$ $\bar{x} \Rightarrow \dots \Rightarrow x$ nicht erfüllbar.

$\Rightarrow$  kann nicht existieren.

„$\Leftarrow$" Betrachte die starken Zusammenhangskomponenten $S_1 \dots S_k$ von $b\varphi$, welche eine Partition von V bilden ($S_1 \dot\cup \dots \dot\cup S_k = V$, $S_i \cap S_j = \emptyset$ $\forall i \neq j$)

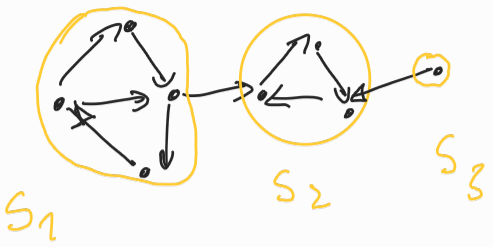


innerhalb einer S_i müssen alle Literale entweder TRUE oder FALSE sein, wenn φ erfüllbar ist.

wegen Ann „$\Leftarrow$" gibt es in S_i nicht x & $\bar{x}$ gleichzeitig!

(kann man in poly. Zeit testen)

betrachte jetzt „Quotientengraph" $H = G_\varphi / \leftrightarrow$



mit Knoten $S_1 \dots S_k$
& kanten (S_i, S_j)
wenn es ein
$l \in S_i$ & $l' \in S_j$
gibt od $(l, l') \in E(G_\varphi)$.

$\circ S_1 \rightarrow \circ S_2 \leftarrow \circ S_3$

H ist kreisfrei $\Rightarrow$ in H gibt es topologische Sortierung $<$

d.h. $S_i < S_j$ wenn (S_i, S_j) kante ist.

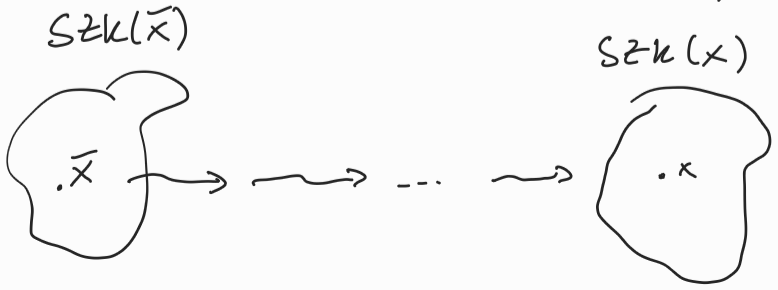
Im Folgenden sei $SZK(l)$ die Zus.hg.k. die l enthält
($l \in \{x, \bar{x}\}$ für eine variable x)

per Ann.: $SZK(x) \neq SZK(\bar{x})$

$\Rightarrow$ in topolog. sortierung: $<$ oder $>$ (statt $\neq$)

Regel ⊛: setze $x = \text{TRUE} \iff SZK(\bar{x}) < SZK(x)$

(f.a. variablen kommt x & $\bar{x}$ in G_φ vor)



würden wir φ FALSE sehen,
$\Rightarrow$ Kette von Implikationen

$\bar{x}$ $\Rightarrow$ .... $\Rightarrow$ x $\Rightarrow \varphi$ FALSE
TRUE FALSE ↯

(analog $Szk(x) < Szk(\bar{x})$ dann x = FALSE
$x \Rightarrow \dots \rightarrow \bar{x}$ $\Rightarrow \bar{x}$ = TRUE)

Warum klappt das?

Ann: es klappt nicht, d.h. φ nicht erfüllt durch diese Belegung
$\Rightarrow$ „Implikation, $l \Rightarrow l'$"
d.h. l = TRUE & l' = FALSE

$\Rightarrow$ $Szk(\bar{l}) < Szk(l)$
$Szk(l') < Szk(\bar{l})$

Regel ⇔ bzgl topol. Ordnung $<$

in G_φ existieren Kanten $l \longrightarrow l'$ & $\bar{l}' \longrightarrow \bar{l}$

$\Rightarrow$ in H: $Szk(l) \leq Szk(l')$
& $Szk(\bar{l}') \leq Szk(\bar{l})$

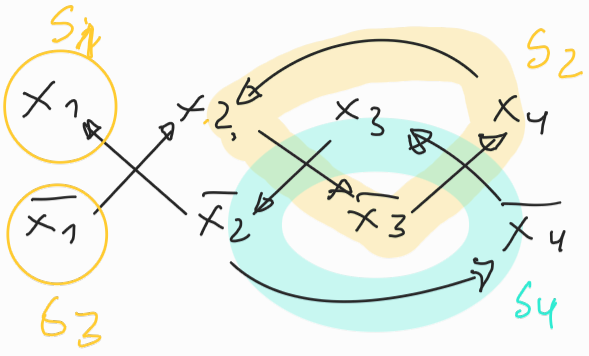
$\Rightarrow$ $Szk(l) \leq Szk(l') < Szk(\bar{l}') \leq Szk(\bar{l}) < Szk(l)$ ↯

$\Rightarrow$ haben Ann φ nicht erfüllt zum Widerspruch geführt
$\Rightarrow$ φ ist erfüllt.

Die letzten beiden Schritte können in linearer Zeit implementiert werden (Übungsaufg.). □

Bsp: $(x_1 \vee x_2) \wedge (x_2 \vee \bar{x}_4) \wedge (\bar{x}_2 \vee \bar{x}_3) \wedge (x_3 \vee x_4)$

Gφ:



H: $S_1 \to S_2$, $S_3 \to S_4$

sortiert bspw.:

$S_4 < S_3 < S_2 < S_1$

$\bar{x}_1$ x_1

$\bar{x}_2$ x_2

x_3 $\bar{x}_3$

$\bar{x}_4$ x_4

REGEL(φ): $x = \text{TRUE} \iff Szk(\bar{x}) < Szk(x)$

$\Rightarrow$

x_1 TRUE

x_2 TRUE

x_3 FALSE

x_4 TRUE

$(x_1 \vee x_2) \wedge (x_2 \vee \bar{x}_4) \wedge (\bar{x}_2 \vee \bar{x}_3) \wedge (x_3 \vee x_4)$ ✓

1 1 1 0 0 1 0 1

Was ist mit 3SAT?

Bisher wurde kein poly-Alg zum Lösen von 3SAT gefunden → gibt es eins?

Eulertouren

Def: Multigraph $G = (V, E)$, Kanten können mehrfach auftreten

bsp:

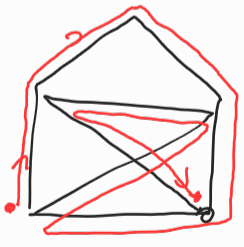


Def: Sei $G = (V, E)$ ein endlich ungerichteter Multigraph

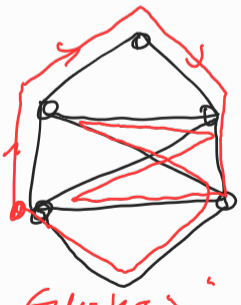
Ein Eulerweg ist Pfad der jede Kante von G genau einmal enthält

(d.h. Knoten können mehrmals durchlaufen werden, also kein einfacher Pfad)

Eulertour ist ein VV-Pfad der ein Eulerweg ist, d.h. Start & Endknoten stimmen überein.



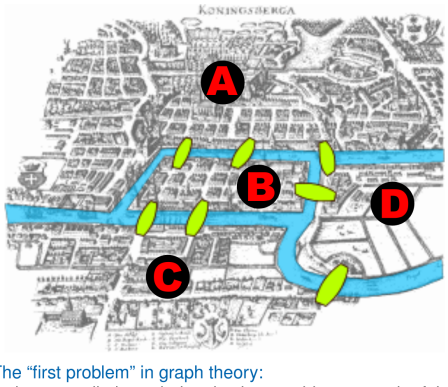
„Eulerweg"

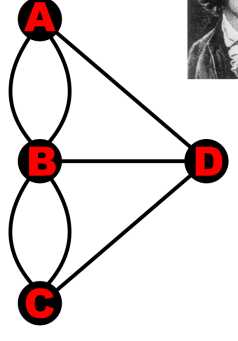


„Eulerkreis"

[Hamilt. Kreis: jeder Knoten genau einmal.
Eulerkreis: jede Kante genau einmal]

Seven Bridges of Königsberg (Euler, 1736)







The "first problem" in graph theory:
Is there a walk through the city that would cross each of those bridges once and only once.
Original article: https://scholarlycommons.pacific.edu/euler-works/

Thm (Euler, 1736)

Ein zshgd. (Multi) graph G besitzt eine Eulertour

$\Leftrightarrow$ $\forall v \in V(G)$: $\deg(v)$ ist gerade.

$\Rightarrow$ Entscheidung ob Eulertour exist: check ob alle Knotengrade gerade sind.

Das Bestimmen einer Eulertour ist dann auch in poly zeit lösbar.

Was wenn wir jetzt Touristen die, anstatt jeder Kante, jeden Knoten genau einmal besucht?

(aka: Hamiltonsche Kreis)

Bisher kein poly. zeit Alg gefunden der dieses Problem löst!

Gibt es solch einen Alg?

Zusammenfassung

Haben Probleme betrachtet,
die algorithmisch einfach sind, d.h.
in pol. Zeit gelöst werden können

- kürzester Weg in Graphen
- 2SAT
- Euler-Touren

Zu allen Problemen gibt es "minimale" Abwandlungen
für die bisher kein polyzeit Alg. gefunden wurde.

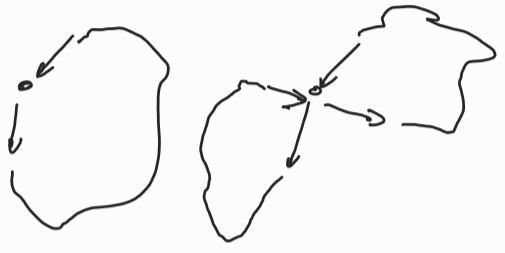
⟶ WAS NUN?

Anfang

Wann existiert Eulertour?

Thm (Euler, 1736)
Ein zush.hgd. (Multi-) graph G besitzt eine Eulertour
$\Leftrightarrow \forall v \in V(G) : \deg(v)$ ist gerade.

Beweis: „$\Rightarrow$":



usw.

jedes mal wenn Tour auf Knoten v trifft, muss v über noch nicht benutzte Kante verlassen werden, d.h. die zu v inzidenten Kanten treten in Paaren auf (1 zum reingehen in v & 1 zum rausgehen von v)

$\Rightarrow \deg(v)$ gerade.

„$\Leftarrow$" Ann. $\forall v \in V : \deg(v)$ gerade.
Wähle bel. Startknoten s aus G
& gehe entlang unbenutzter Kanten bis s wieder erreicht wird.

das klet, weil



unbenutzt da $\deg(v)$ gerade muss eine Ausgangskante existieren
sobald wir v (wieder) besuchen.

$\Rightarrow$ Pfad W kann nur in s enden

jetzt schaut man ob alle Kanten benutzt wurden.

wenn nicht betrachte, betrachte G' der aus G durch entfernen aller benutzten Kanten in W entsteht.

$\Rightarrow$ an jeden Knoten wird eine gerade Anzahl (0 eventuell) von Kanten entfernt.

Da die Knotengrade gerade waren, sind auch in G' die Knotengrade gerade.

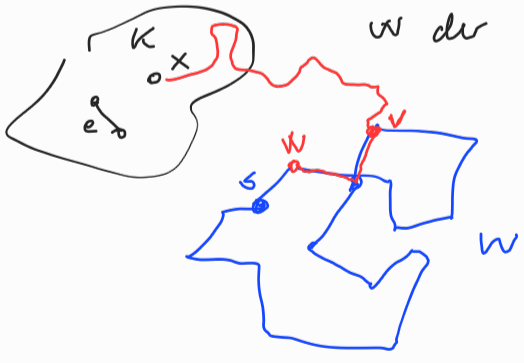
da wir annahmen daß es noch unbesuchte Kanten gibt $\Rightarrow$ G' enthält mind 1 Kante e

Sei K die Zusammenhangskomponente in G' die e enthält.

Wir zeigen jetzt daß K einen Knoten enthält der auch in W liegt.

Wähle $x \in V(K)$.

da G zusammenh. $\Rightarrow$ exist ein Pfad P' in G von x zu einem Knoten w der in W liegt.



Sei v der erste Knoten von P' der auf W liegt.

$\Rightarrow$ alle Kanten auf dem Teilpfad von P' von x zu v sind unbenutzt.

$\Rightarrow$ v liegt auch in K

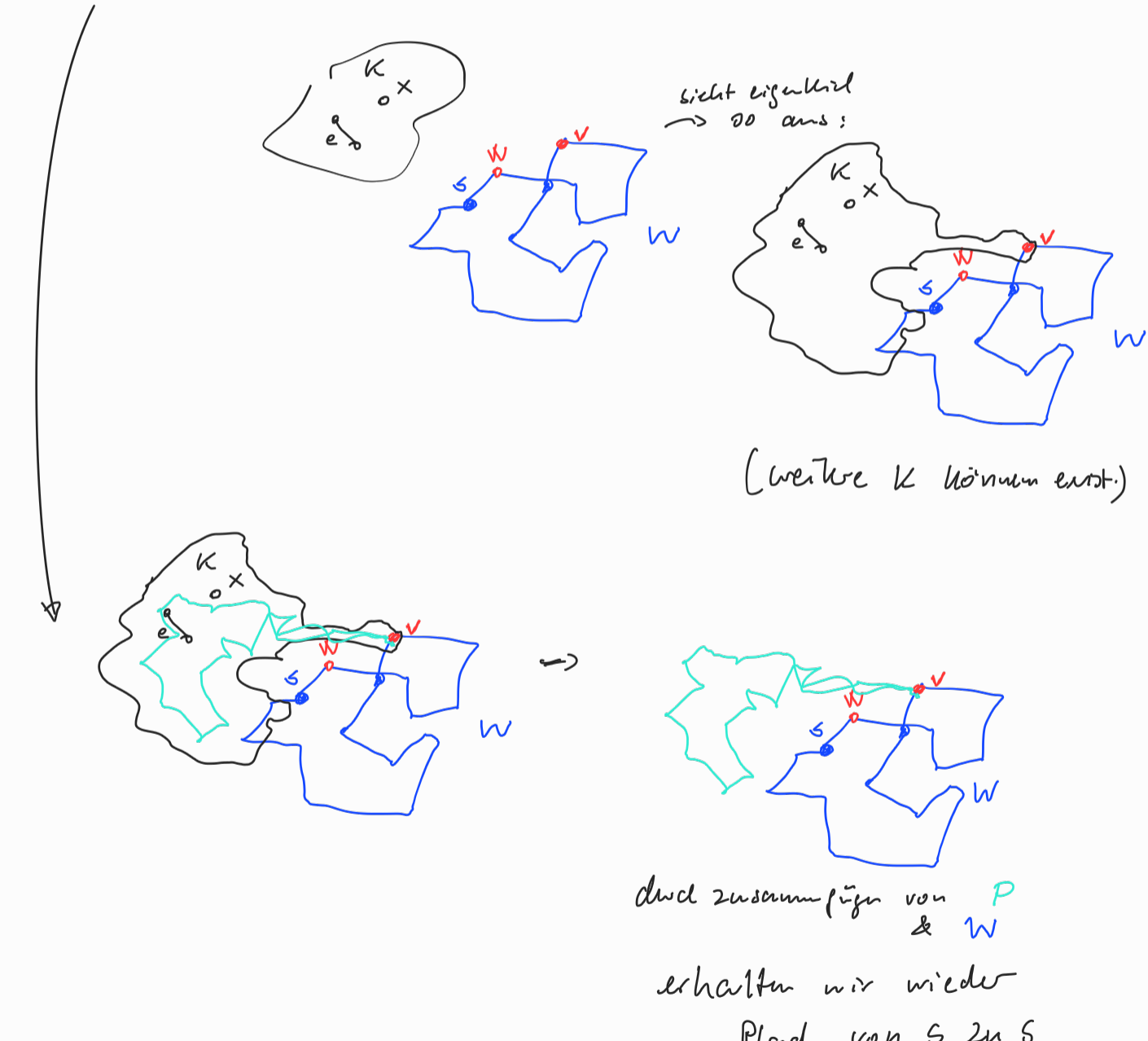
⇒ es gibt keinen Weg von w zu v

k abhängig von Weigen.

d.h. in G' gibt es keinen Weg von x zu v

d das gilt f.a. zush. Komponenten K.

können jetzt mit v als Startknoten, beginnend einen vv-Pfad in G' suchen



wdh. Widerspruchsargument, da alle Knoten besucht.

□

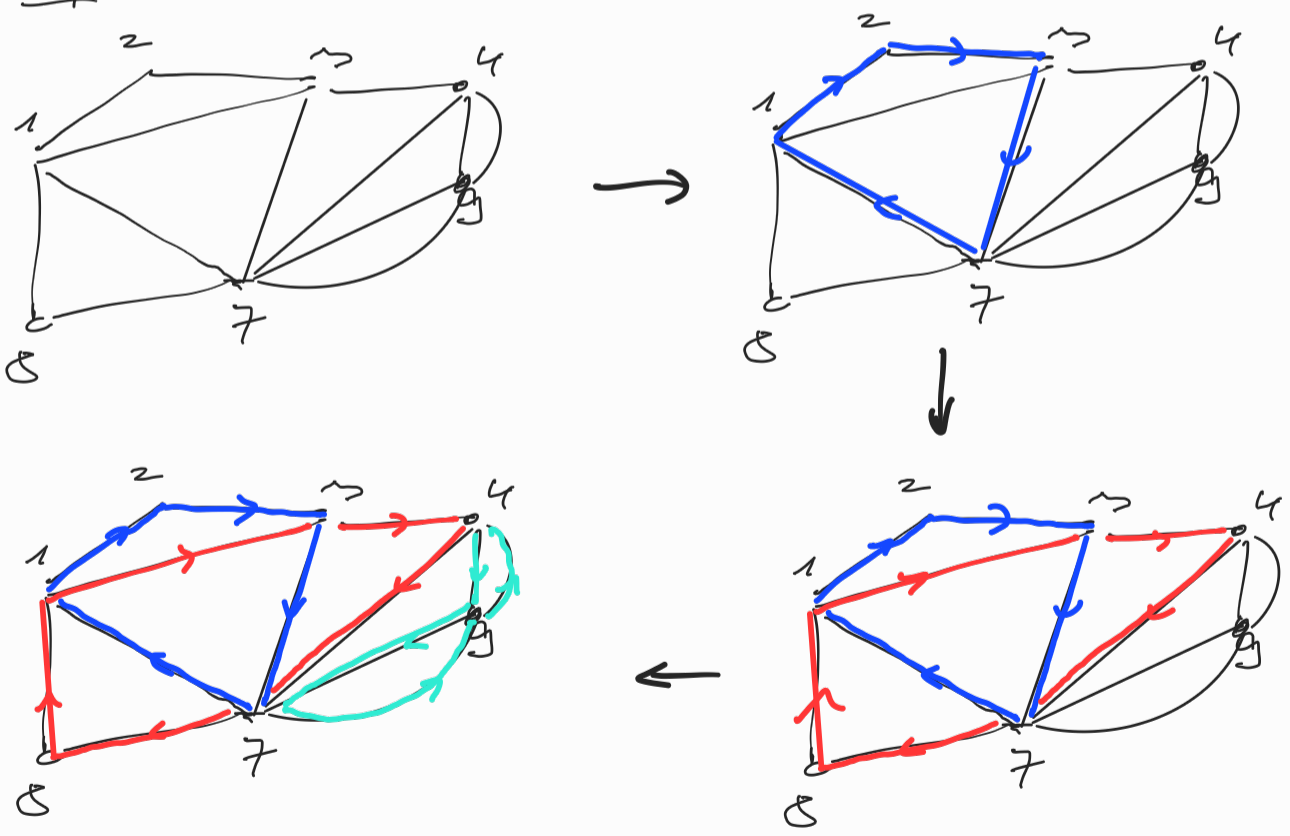
A3

[auch bekannt als Hierholzers Alg (1873)]

Alg - Eulertour (G)

wähle $s \in V(G)$ beliebig
Folge unbesuchten Kanten bis s wieder erreicht
Sei W der Pfad von s zu s
WHILE (W enthält Knoten v, der inzident zu unbesuchter Kante ist)
- Starte in v &
- Folge unbesuchten Kanten bis v wieder erreicht
- Füge den neuen geschlossen Rundgang zu W hinzu

Bsp:



Thm: Hierholzers Alg berechnet Eulertour in $O(|V| + |E|)$ f.a zusammenh. Multigraphen $G = (V, E)$ deren Knotengrade gerade sind.

Beweis: Korrektheit (s.o.)
Laufzeit (ü.A.)

$\Rightarrow$ Entscheidung ob Eulertour existiert: check ob alle Knotengrade gerade sind.

Das Bestimmen einer Eulertour ist dann auch in Polyzeit lösbar.

AS